

11b — Microservices

Key Concepts

- **Microservice** — small, independently deployable service that does one thing and owns its own data
- **Database per service** — no shared databases; each service owns its data
- **API Gateway** — single entry point that routes requests to correct services
- **Saga pattern** — sequence of local transactions with compensating actions on failure
- **Correlation ID / Trace ID** — unique ID attached to every request, forwarded across all services for tracing
- **Compensating transaction** — undo action triggered when a downstream step in a Saga fails

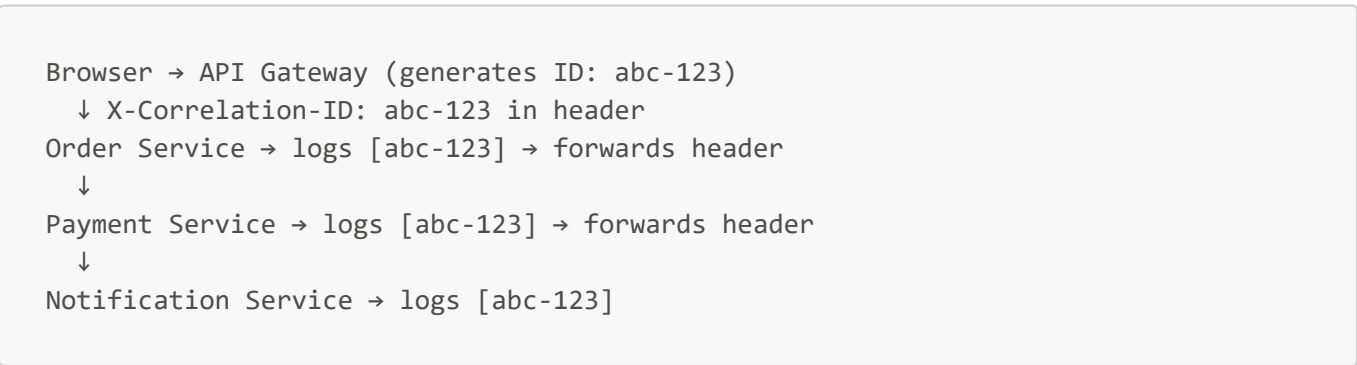
How It Works



Communication Styles

Style	How	When to Use
Synchronous	REST / gRPC — caller waits	Need immediate answer
Asynchronous	Message queue (Kafka, RabbitMQ, Service Bus) — fire and forget	Caller doesn't need to wait

Correlation ID Flow



- Added to **request header** — so downstream services read and forward it
- Added to **response header** — so browser/client has it for support and debugging
- For async (Service Bus) — embedded in message body/properties

Strengths vs Weaknesses

Strength	Weakness
Independent deployment	Distributed systems complexity
Independent scaling	Data consistency across services
Fault isolation	Observability — tracing across services
Technology flexibility	Testing requires multiple services running
Team autonomy	Operational overhead — 12 services = 12 deployments

Data Consistency Problem

- No shared DB = no ACID transactions across services
- **Saga pattern** solves this:
 - Each service completes local transaction
 - If downstream fails → compensating transaction rolls back previous steps
 - Example: payment fails → Saga triggers cancel-order in Order Service

Industry Examples

- **Netflix** — 700+ microservices; teams deploy hundreds of times daily independently
- **Uber** — split from monolith after dispatch bug took down driver payments; microservices gave fault isolation
- **Amazon** — "two-pizza team" rule: each team owns one service end-to-end

Interview Questions

Q: What is a microservices architecture? A: An approach where the app is split into small, independently deployable services, each owning its own data and communicating over a network.

Q: What's the biggest challenge with microservices? A: Data consistency — without a shared database, maintaining consistency requires patterns like Saga, which adds significant complexity.

Q: How do microservices communicate? A: Synchronously via REST or gRPC when an immediate response is needed; asynchronously via message queues (Kafka, RabbitMQ) when the caller doesn't need to wait.

Q: When should you NOT use microservices? A: Early stage — when the domain isn't understood yet, you'll draw wrong service boundaries and end up with a distributed monolith.

Q: What is a Correlation ID and why is it added to the response? A: A unique ID that travels with every request across all services for tracing. Added to response so the browser/client has it — support teams can search one ID in logs to see the full request journey.